

PROGRAMS ON STL

~Mishti Choudhari

//Program 1: Array Sorting using begin() and end() #include <algorithm>

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int arr[] = {3, 5, 1, 2, 4};
```

```
// Sort the array in ascending order
```

```
// std::sort(std::begin(arr), std::end(arr));
```

```
    sort(begin(arr), end(arr));
```

```
// Print the sorted array
```

```
    for (int i : arr) {
```

```
        cout << i << " ";
```

```
    }
```

```
    return 0;
```

```
}
```

Output: 1 2 3 4 5

//Program 2: Array Sorting using Pointer Range (arr + size)

```
#include <iostream>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
void show(int a[],int array_size) {
```

```
    for(int i=0;i<array_size;i++) {
```

```
        cout<<a[i]<<" ";
```

```
    }
```

```
}
```

```
int main() {
```

```
    int arr[]={1,5,8,9,6,7,3,4,2,0};
```

```
//Size of array
```

```
    int asize=sizeof(arr)/sizeof(arr[0]);
```

```

    cout<<"The array before sorting is:\n";
    show(arr,asize); //Print array
    sort(arr,arr+asize); //Sort array
    cout<<"\n\nThe array after sorting is:\n";
    show(arr,asize);
    return 0;
}

```

Output: The array before sorting is:

1 5 8 9 6 7 3 4 2 0

The array after sorting is:

0 1 2 3 4 5 6 7 8 9

//Program 3: Stack sorting/searching using stl

```

#include <iostream>
# include<stack>
using namespace std;
int main() {
    stack<int> stack;
    stack.push (21);
    stack.push (22);
    stack.push (24);
    stack.push (25);
    cout<<stack.top();
    int num = 0;
    stack.push (num);
    stack.pop();
    stack.pop();
    while(!stack.empty()) {
        cout<<stack.top()<<" ";
        stack.pop();
    }
    return 0;
}

```

```
}
```

Output: 25 24 22 21

//Program 4: Queue sorting/searching using stl

```
#include <iostream>
```

```
#include <queue>
```

```
using namespace std;
```

```
// Print the queue
```

```
void showq(queue<int> gq) {
```

```
    queue<int> g = gq;
```

```
    while (!g.empty()) {
```

```
        cout << '\t' << g.front();
```

```
        g.pop();
```

```
    }
```

```
    cout << '\n';
```

```
}
```

```
// Driver Code
```

```
int main() {
```

```
    queue<int> que;
```

```
    que.push(11); // [11, 22, 33]
```

```
    que.push(22);
```

```
    que.push(33);
```

```
    cout << "The queue que is : ";
```

```
    showq(que);
```

```
    cout << "\n que.size(): " << que.size();
```

```
    cout << "\n que.front(): " << que.front();
```

```
    cout << "\n que.back(): " << que.back();
```

```
    cout << "\n que.pop(): ";
```

```
    que.pop();
```

```
    showq(que);
```

```
    return 0;
```

```
}
```

Output: The queue que is : 11 22 33

que.size(): 3

que.front(): 11

que.back(): 33

que.pop(): 22 33

//Program 5: □ Special Character Counter in String

```
#include<iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    string str;
```

```
    cin>>str;
```

```
    int count=0;
```

```
    for(int i=0; i<str.length(); i++) {
```

```
        if(!((str[i]>='a' && str[i]<='z') &&
```

```
            !(str[i]>='A' && str[i]<='Z') &&
```

```
            !(str[i]>='0' && str[i]<='9'))
```

```
        {
```

```
            count++;
```

```
        }
```

```
    }
```

```
    cout<<count;
```

```
    return 0;
```

```
}
```

//Program 6: Vector Operations using STL

```
#include<iostream>
```

```
#include<vector>
```

```
using namespace std;
```

```
int main() {
```

```
    vector<int> g1;//var
```

```
    for (int i=0;i<=5;i++) {
```

```

        g1.push_back(i); // push 1-5
    }
    cout<<"Output of begin and end: ";
    for(auto i=g1.begin();i!=g1.end();i++){
        cout<<*i<<" "; //Value at i
    }
    cout<<"\nOutput of rbegin and rend: ";
    for(auto ir=g1.rbegin();ir != g1.rend();++ir) { //rightbeign
        cout<<*ir<<" ";
    }
    cout<<"\nSize:"<<g1.size();
    cout<<"\nCapacity:"<<g1.capacity();
    cout<<"\nMax_Size:"<<g1.max_size();
    cout<<"\nat \t: g1.at(4)="<<g1.at(3);
    cout<<"\nfront \t: g1.front(4)="<<g1.front();
    cout<<"\nback \t: g1.back(4)="<<g1.back();
    return 0;
}

```

Output: Output of begin and end: 0 1 2 3 4 5

Output of rbegin and rend: 5 4 3 2 1 0

Size:6

Capacity:8

Max_Size:2305843009213693951

at : g1.at(4)=3

front : g1.front(4)=0

back : g1.back(4)=5

//Program 7: Switch Case without Break (Fall Through)

```
#include<iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    int i=1;
```

```
switch(i) {  
    case 1:  
        cout<<"Hi"<<endl;  
    default:  
        cout<<"Hey";  
}  
return 0;  
}
```

Output: Hi Hey

//Program 8: Character-Based Switch Case

```
#include<iostream>  
using namespace std;  
int main(){  
    char ch=65;  
    switch(ch) {  
        case 'A': //65 is ascii code for A  
            cout<<"Apple"<<endl;  
            break;  
        case 'B':  
            cout<<"Bye"<<endl;  
            break;  
        default:  
            cout<<"Hey";  
    }  
    return 0;  
}
```

Output: Apple

//Program 9: Duplicate Case Value Error in Switch

```
#include<iostream>  
using namespace std;
```

```

int main(){
    int i=65;
    switch(i) {
        case 65:
            cout<<"int"<<endl;
            break;
        case 'A': //65 is ascii code for A
            cout<<"char"<<endl;
            break;
    }
    return 0;
}

```

Output: error: duplicate case value

//Program 10: Comma Operator in Switch Statement

```

#include<iostream>
using namespace std;
int main()
{
    int i=65;
    char ch='B';
    switch(ch,i) { //The value of the whole expression is the value of expression2 i.e i=65
        case 65:
            cout<<"int"<<endl;
            break;
        case 'B': //65 is ascii code for A
            cout<<"char"<<endl;
            break;
    }
    return 0;
}

```

Output: int

//Program 11: Post-Decrement Evaluation in Switch

```
#include<iostream>
using namespace std;
int main(){
    int i=1;
    i++;
    switch(i--) { //inc i=2, postdec
        case 1:
            cout<<"1"<<endl;
            break;
        case 2:
            cout<<"2"<<endl;
            break;
    }
    return 0;
}
```

Output: 2

//Program 12: Switch Case Fall Through Example

```
#include<iostream>
using namespace std;
int main(){
    switch(3/2) { //1.5=1 and no break after case1
        case 1:
            cout<<"1"<<endl;
        case 2:
            cout<<"2"<<endl;
            break;
        default:
            cout<<"Default";
    }
}
```



```
    return 0;
}
```

Output: 1 2

//Program 13: Comma Operator in Conditional Expression

```
#include<iostream>
using namespace std;
int main(){
    int i=0;
    if(i++,i--,++i) //0;1;0;1
    switch(i) {
        case 1:
            cout<<"1"<<endl;
            break;
        case 2:
            cout<<"2"<<endl;
            break;
        default:
            cout<<"Default";
    }
    return 0;
}
```

Output: 1

//Program 14: Boolean Expression in Switch Statement

```
#include<iostream>
using namespace std;
int main(){
    int i=5;
    switch(i++ == 5) { //Postinc 5==5 True=1
        case 1:
```

```

        cout<<"True"<<endl;
        break;
    case 0:
        cout<<"False"<<endl;
        break;
    default:
        cout<<"Default";
    }
    return 0;
}

```

Output: True

//Program 15: Number Base Conversion using Manipulators

```

#include<iostream>
using namespace std;
int main() {
    int a=10;
    cout<<"Hex="<<hex<<a<<endl;
    cout<<"Oct="<<oct<<a<<endl;
    cout<<"Dec="<<dec<<a<<endl;
    return 0;
}

```

o/p: Hex=a

Oct=12

Dec=10

//Program 16: Field Width Formatting using setw()

```

#include<iostream>
#include <iomanip> //Manipulator
using namespace std;
int main() {
    int a=200;

```

```

    cout<<setw(8)<<a<<endl; //setw
    cout<<setw(5)<<a<<setw(6)<<a<<endl;
    cout<<setw(3)<<a<<setw(10)<<a<<endl;
return 0;
}
o/p: 200
      200 200
200    200

```

//Program 17: Output Formatting using setfill() and setw()

```

#include<iostream>
#include <iomanip>
using namespace std;
int main() {
    int a=200;
    cout<<setfill('_');
    cout<<setw(8)<<a<<endl;
    cout<<setw(5)<<a<<setw(6)<<a<<endl;
    cout<<setw(3)<<a<<setw(10)<<a<<endl;
return 0;
}
o/p: ____200
      _200_200
200_____200

```